PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR

Facultad de Hábitat, Infraestructura y Creatividad • Programación Móvil (API 36)

Informe de Laboratorio 5.2 / 5.3: Reto Integrador
Implementación de un Termostato Inteligente Reactivo

Asignatura:	Redes y Dispositivos Avanzados 2 (RDA-2) / Programación Móvil	**Fecha:**	23 de Octubre de 2026
Docente:	Dr. Juan Francisco Chafla, PhD.	**Entorno:**	Android Studio Ladybug | API 36 | Jetpack Compose
Grupo:	Reto Grupo 1	**Commit Final:**	`e0b199a`
Estudiante:	Paul Rosero	**Repositorio:**	`github.com/paulrosero/RDA2_Lab5.3_Reto_Grupo1`

1. Objetivo del Reto y Contextualización

El presente reto integrador tiene como propósito aplicar los fundamentos del **paradigma declarativo** y la **arquitectura reactiva** impartidos en el *Tema 5*, tales como la **Gestión de Estado**, **State Hoisting (Elevación de Estado)**, **Flujo Unidireccional de Datos (UDF)** y la **Recomposición Inteligente**, para diseñar un panel de climatización funcional en **Android API 36**.

A diferencia del paradigma imperativo tradicional (layouts XML y Java/Kotlin), donde los elementos visuales se actualizaban manualmente mediante `findViewById`, `setText()` y `setVisibility()` propiciando fallas de desincronización, en Jetpack Compose la interfaz se concibe como una función determinista directa del estado: **UI = f(State)**. El sistema redibuja de forma instantánea la apariencia gráfica basándose exclusivamente en el flujo de datos.

2. Marco Teórico y Buenas Prácticas (Tema 5)

> **Principio Filosófico "Cómo vs Qué":** En lugar de describir paso a paso la secuencia de instrucciones para alterar los widgets existentes (paradigma imperativo), el desarrollador describe el estado visual final deseado (paradigma declarativo). Compose gestiona la ejecución y optimización de forma automática.

Concepto	Paradigma Tradicional (XML)	Paradigma Declarativo (Jetpack Compose)
Estructura	Archivos XML estáticos separados de la lógica.	Funciones `@Composable` con DSL especializado en Kotlin.
Sincronización	Manual mediante `findViewById` y setters.	Automática y reactiva ante cambios en las variables de estado.
Fuente de Verdad	Dispersa entre variables y propiedades de vistas.	Centralizada (Single Source of Truth mediante State Hoisting).
Rendimiento	Inflado de jerarquías de vistas pesadas en memoria.	Recomposición inteligente con omisión (skipping) de nodos estáticos.
Convergencia	Android View System (dispar a iOS UIKit).	Alineado con SwiftUI en iOS (`Column`/`VStack`, `remember`/`@State`).

Pilares Clave de Implementación:

- **`remember`:** Preserva el valor en memoria a través de las sucesivas recomposiciones del árbol de composición.
- **`mutableIntStateOf`:** Observador reactivo primitivo de alto rendimiento que notifica a Compose las mutaciones del entero de temperatura sin incurrir en boxing innecesario.
- **State Hoisting (Elevación del Estado):** El estado mutable reside únicamente en el contenedor padre (`ThermostatScreen`), convirtiendo a los subcomponentes hijos en **Stateless** (puros, altamente testeables y reutilizables).
- **Unidirectional Data Flow (UDF):** El estado desciende por parámetros inmutables (*State Down*) y las acciones del usuario ascienden hacia el padre mediante funciones lambda (*Events Up*).

3. Jerarquía y Arquitectura de la Solución

```
[ThermostatScreen (Stateful Parent - Single Source of Truth)]
    │
    ├──► State Down (temperature: Int) ──────► [TemperatureDisplay (Stateless Child)]
    │                                                ├─ Lógica Condicional: temp >= 25°C ? Rojo : Azul
    │                                                └─ Icono: WbSunny (Sol) o AcUnit (Nieve)
    │
    └──◄ Events Up: onIncrease() / onDecrease() ◄ [TemperatureControls (Stateless Child)]
                                                  └─ Botones delegados: Subir (+) y Bajar (-)
```

Cumpliendo rigurosamente con los **Requisitos Técnicos**:

- Se prescindió en su totalidad de archivos XML de layout o selectores estáticos.
- Se utilizaron modificadores `Modifier.padding()` y elementos `Spacer` para mantener márgenes y jerarquías proporcionales.
- El componente `TemperatureDisplay` no posee ningún estado mutable interno, satisfaciendo la condición de ser 100% *Stateless*.

4. Evidencias de Ejecución y Validación en Emulador API 36

La aplicación fue validada en el emulador **Pixel 8 (Android API 36)**. Al pulsar los botones de interacción, la apariencia visual (número, etiqueta de modo, color de acento, fondo de tarjeta e icono) conmutó de forma inmediata y automática sin recarga de Activity.

ESTADO FRÍO (< 25°C)



Temperatura: 20°C (Valor Inicial)
Icono de Copo de Nieve (AcUnit) • Tonalidad Azul (#1976D2) • Modo Refrigeración

ESTADO CALOR (>= 25°C)



Temperatura: 26°C (Umbral Superado)
Icono de Sol (WbSunny) • Tonalidad Roja (#D32F2F) • Modo Calefacción

5. Análisis de Resultados y Discusión Técnica

1. **Reactividad Determinista:** La condición `val isWarm = temperature >= 25` dentro de `TemperatureDisplay` define de manera concisa el comportamiento visual completo. Toda mutación sobre `temperature` generada en los controles actualiza los nodos correspondientes en menos de 16 milisegundos (60-120 fps).
2. **Eficiencia por Recomposición Inteligente:** Al comparar con la analogía de la hoja de cálculo explicada en clase, Compose redibujó exclusivamente el bloque interno de `TemperatureDisplay`, omitiendo por completo los títulos estáticos de la cabecera.
3. **Desacoplamiento Estructural:** Los botones no alteran variables de forma directa; delegan la mutación a través de lambdas al contenedor padre, garantizando una trazabilidad perfecta del estado y permitiendo crear vistas previas (`@Preview`) de forma independiente.

6. Conclusiones

1. **Consolidación del Paradigma Declarativo:** La implementación demostró cómo Jetpack Compose simplifica el desarrollo móvil al convertir la UI en una función directa del estado, eliminando errores de desincronización

propios de XML.

2. **Madurez de la Arquitectura con State Hoisting:** Centralizar el estado en el componente padre y delegar a hijos Stateless asegura un código desacoplado, modular y óptimo para pruebas unitarias e instrumentadas.
3. **Alineación con Estándares de la Industria en API 36:** La utilización de Kotlin 2.0 y Compose en API 36 valida que los conocimientos adquiridos en el Tema 5 corresponden a las mejores prácticas de ingeniería de software vigentes a nivel global.

7. Recomendaciones

1. **Optimización de tipos de datos:** Preferir `mutableIntStateOf` en lugar de `mutableStateOf<Int>` para prevenir la sobrecarga de boxing/unboxing en contadores de alta frecuencia.
2. **Uso estricto del Bill of Materials (BOM):** Mantener alineadas las dependencias de Material Icons Extended mediante `platform(libs.androidx.compose.bom)` para garantizar compatibilidad binaria.
3. **Diseño basado en Spacers y Modifiers:** Emplear espaciadores relativos y modificadores de relleno en lugar de tamaños absolutos fijos para garantizar una interfaz responsiva ante múltiples formatos de pantalla.

8. Anexo: Código Fuente Implementado (MainActivity.kt)

A continuación se presenta el código fuente íntegro del archivo `MainActivity.kt` con comentarios explicativos en cada una de sus secciones:

```
package com.paulrosero.laboratorio1

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.BorderStroke
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.AcUnit
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.Remove
import androidx.compose.material.icons.filled.WbSunny
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.paulrosero.laboratorio1.ui.theme.Laboratorio1Theme

/**
 * LABORATORIO 5.2 / 5.3: RETO INTEGRADOR - CLIMATIZADOR REACTIVO
 * Demostración práctica de UI Declarativa, State Hoisting y Recomposición Inteligente.
 */
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge() // Habilita dibujo edge-to-edge en API 36
        setContent {
            Laboratorio1Theme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    ThermostatScreen(
                        modifier = Modifier.fillMaxSize().padding(innerPadding)
                    )
                }
            }
        }
    }
}

/**
 * COMPONENTE PADRE (Stateful): Contenedor principal y única fuente de verdad (Single Source of Truth).
 * Gestiona el estado reactivo con remember y mutableIntStateOf(20).
 */
@Composable
fun ThermostatScreen(modifier: Modifier = Modifier) {
    // Estado reactivo: valor entero inicial en 20°C
    var temperature by remember { mutableIntStateOf(20) }

    Column(
        modifier = modifier
            .fillMaxSize()
            .background(MaterialTheme.colorScheme.background)
            .padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
```

```
        Text(
            text = "Climatizador Reactivo",
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold
        )
        Text(
            text = "Control de Climatización Inteligente - API 36",
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )

        Spacer(modifier = Modifier.height(32.dp))

        // State Down: El estado desciende hacia el hijo de visualización
        TemperatureDisplay(
            temperature = temperature,
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(modifier = Modifier.height(32.dp))

        // Events Up: Los eventos de usuario ascienden al padre mediante lambdas
        TemperatureControls(
            onIncrease = { temperature++ },
            onDecrease = { temperature-- },
            modifier = Modifier.fillMaxWidth()
        )
    }
}

/**
 * SUBCOMPONENTE DE VISUALIZACIÓN (Stateless):
 * Recibe la temperatura como parámetro. No almacena estado interno.
 * Aplica lógica condicional declarativa: >= 25°C -> Rojo y Sol; < 25°C -> Azul y Nieve.
 */
@Composable
fun TemperatureDisplay(temperature: Int, modifier: Modifier = Modifier) {
    val isWarm = temperature >= 25
    val statusColor = if (isWarm) Color(0xFFD32F2F) else Color(0xFF1976D2)
    val backgroundColor = if (isWarm) Color(0xFFFFEBEE) else Color(0xFFE3F2FD)
    val icon = if (isWarm) Icons.Filled.WbSunny else Icons.Filled.AcUnit
    val statusLabel = if (isWarm) "Modo Calor (>= 25°C)" else "Modo Frío (< 25°C)"
    val descriptionText = if (isWarm) "Calefacción Activada" else "Refrigeración Activada"

    Card(
        modifier = modifier,
        shape = RoundedCornerShape(24.dp),
        colors = CardDefaults.cardColors(containerColor = backgroundColor),
        border = BorderStroke(2.dp, statusColor.copy(alpha = 0.5f)),
        elevation = CardDefaults.cardElevation(defaultElevation = 6.dp)
    ) {
        Column(
            modifier = Modifier.fillMaxWidth().padding(vertical = 36.dp, horizontal = 24.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Box(
                modifier = Modifier
                    .size(100.dp)
                    .clip(CircleShape)
                    .background(statusColor.copy(alpha = 0.15f)),
                contentAlignment = Alignment.Center
            ) {
                Icon(
                    imageVector = icon,
                    contentDescription = statusLabel,
                    tint = statusColor,
                    modifier = Modifier.size(64.dp)
                )
            }

            Spacer(modifier = Modifier.height(18.dp))

            Text(
```

```
                text = "$temperature°C",
                fontSize = 58.sp,
                fontWeight = FontWeight.ExtraBold,
                color = statusColor,
                textAlign = TextAlign.Center
            )

            Text(
                text = statusLabel,
                fontSize = 18.sp,
                fontWeight = FontWeight.SemiBold,
                color = statusColor
            )

            Spacer(modifier = Modifier.height(6.dp))

            Text(
                text = descriptionText,
                fontSize = 14.sp,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
        }
    }
}

/**
 * SUBCOMPONENTE DE CONTROLES (Stateless):
 * Contiene dos botones (Subir/Bajar) que ejecutan lambdas recibidas del contenedor padre.
 */
@Composable
fun TemperatureControls(
    onIncrease: () -> Unit,
    onDecrease: () -> Unit,
    modifier: Modifier = Modifier
) {
    Row(
        modifier = modifier.padding(horizontal = 8.dp),
        horizontalArrangement = Arrangement.SpaceEvenly,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Button(
            onClick = onDecrease,
            shape = RoundedCornerShape(16.dp),
            colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF1976D2)),
            modifier = Modifier.weight(1f).height(56.dp)
        ) {
            Icon(imageVector = Icons.Filled.Remove, contentDescription = "Bajar")
            Spacer(modifier = Modifier.width(8.dp))
            Text(text = "Bajar", fontSize = 16.sp, fontWeight = FontWeight.Bold)
        }

        Spacer(modifier = Modifier.width(16.dp))

        Button(
            onClick = onIncrease,
            shape = RoundedCornerShape(16.dp),
            colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFD32F2F)),
            modifier = Modifier.weight(1f).height(56.dp)
        ) {
            Icon(imageVector = Icons.Filled.Add, contentDescription = "Subir")
            Spacer(modifier = Modifier.width(8.dp))
            Text(text = "Subir", fontSize = 16.sp, fontWeight = FontWeight.Bold)
        }
    }
}

@Preview(showBackground = true, name = "Estado Frío (< 25°C)")
@Composable
fun ThermostatScreenColdPreview() {
    Laboratorio1Theme {
        Surface {
            Column(modifier = Modifier.fillMaxSize().padding(24.dp), horizontalAlignment = Alignment.CenterHorizontally)
{
                Text("Climatizador Reactivo", style = MaterialTheme.typography.headlineMedium, fontWeight = FontWeight.B
```

```
old)
                Spacer(modifier = Modifier.height(32.dp))
                TemperatureDisplay(temperature = 20)
                Spacer(modifier = Modifier.height(32.dp))
                TemperatureControls(onIncrease = {}, onDecrease = {})
            }
        }
    }
}

@Preview(showBackground = true, name = "Estado Calor (>= 25°C)")
@Composable
fun ThermostatScreenWarmPreview() {
    Laboratorio1Theme {
        Surface {
            Column(modifier = Modifier.fillMaxSize().padding(24.dp), horizontalAlignment = Alignment.CenterHorizontally)
{
                Text("Climatizador Reactivo", style = MaterialTheme.typography.headlineMedium, fontWeight = FontWeight.B
old)
                Spacer(modifier = Modifier.height(32.dp))
                TemperatureDisplay(temperature = 26)
                Spacer(modifier = Modifier.height(32.dp))
                TemperatureControls(onIncrease = {}, onDecrease = {})
            }
        }
    }
}
```